CSE513 Assignment 1

Working website: https://public.emirhanaltunel.com/homeworks/cse513-1/

Git Repo: https://github.com/GtuOdevlerEmirhanAltunel/CSE513-HW1

CPU Scheduling Simulator

A deterministic browser-based simulator for FCFS, SJF, SRTF, and Round Robin.

Student: Emirhan Altunel
Student ID: 26420101003

Report scope. This report presents the scheduling model, eight required algorithm–workload examples, hand calculations, simulator comparisons, three additional test cases, invariant checks, comparative analysis, implementation notes, and an AI-assistance statement.

4 algorithms 8 worked examples 3 custom tests 9 automated tests

Contents

1 Model and calculation method **3**
1.1 Required input workloads . . . 3

2 Worked examples: Workload A **3**
2.1 FCFS on Workload A . . . 3
2.2 SJF on Workload A . . . 4
2.3 SRTF on Workload A . . . 5
2.4 Round Robin (q=2) on Workload A . . . 6

3 Worked examples: Workload B **7**
3.1 FCFS on Workload B . . . 7
3.2 SJF on Workload B . . . 8
3.3 SRTF on Workload B . . . 9
3.4 Round Robin (q=2) on Workload B . . . 10

4 Additional test cases **11**
4.1 Idle CPU test . . . 11
4.2 Simultaneous arrivals and scheduling tie . . . 11
4.3 SRTF preemption and equal-remaining rule . . . 11
4.4 Verified schedule properties . . . 12

5 Analysis **12**
5.1 Which algorithm is best for Workload A? . . . 12
5.2 Round Robin with quantum 1, 2, and 4 . . . 12
5.3 When SJF and SRTF produce the same schedule . . . 13
5.4 Why real operating systems estimate CPU bursts . . . 13
5.5 Starvation under SJF and SRTF . . . 13
5.6 Implementation mistake exposed by testing . . . 13

6 Implementation notes **13**

7 AI assistance statement **13**

1 Model and calculation method

The simulator models one work-conserving CPU, one CPU burst per process, no I/O, and zero-time context switches. Arrival times are nonnegative integers; CPU bursts and the Round Robin quantum are positive integers; and process IDs are unique. Intervals are half-open, so $[2, 5)$ supplies three CPU units.

For every process, the metrics are calculated as follows:

$$\begin{aligned} T &= C - A && \text{(turnaround time)},\\ W &= T - B && \text{(waiting time)},\\ R &= FS - A && \text{(response time)}. \end{aligned}$$

Ties prefer the earlier arrival time and then the earlier position in the input list. SRTF preempts only when another ready process has a *strictly* shorter remaining time. At a Round Robin quantum boundary, arrivals at that exact time enter the ready queue before the unfinished running process is re-enqueued.

1.1 Required input workloads

Table 1: Workloads supplied in the assignment.

Workload A			**Workload B**		
Process	Arrival	Burst	Process	Arrival	Burst
P1	0	8	P1	2	3
P2	1	4	P2	2	1
P3	2	2	P3	8	2
P4	4	1	P4	9	1

2 Worked examples: Workload A

2.1 FCFS on Workload A

Input: P1=(0, 8), P2=(1, 4), P3=(2, 2), P4=(4, 1), where each pair is (arrival, burst).

Hand decision trace. P1 is the only process ready at time 0 and runs to completion. At time 8, P2, P3, and P4 are ready. FCFS chooses them by arrival order: P2, P3, then P4.

```
0-8 P1 | 8-12 P2 | 12-14 P3 | 14-15 P4
```

Table 2: FCFS Workload A: hand calculation and simulator output (identical).

Process	A	B	FS	C	T	W	R
P1	0	8	0	8	8	0	0
P2	1	4	8	12	11	7	7
P3	2	2	12	14	12	10	10
P4	4	1	14	15	11	10	10
			Average		**10.50**	**6.75**	**6.75**

Simulator comparison. The simulator produced the same timeline and every per-process value in the table. **Exact match; no discrepancy.**

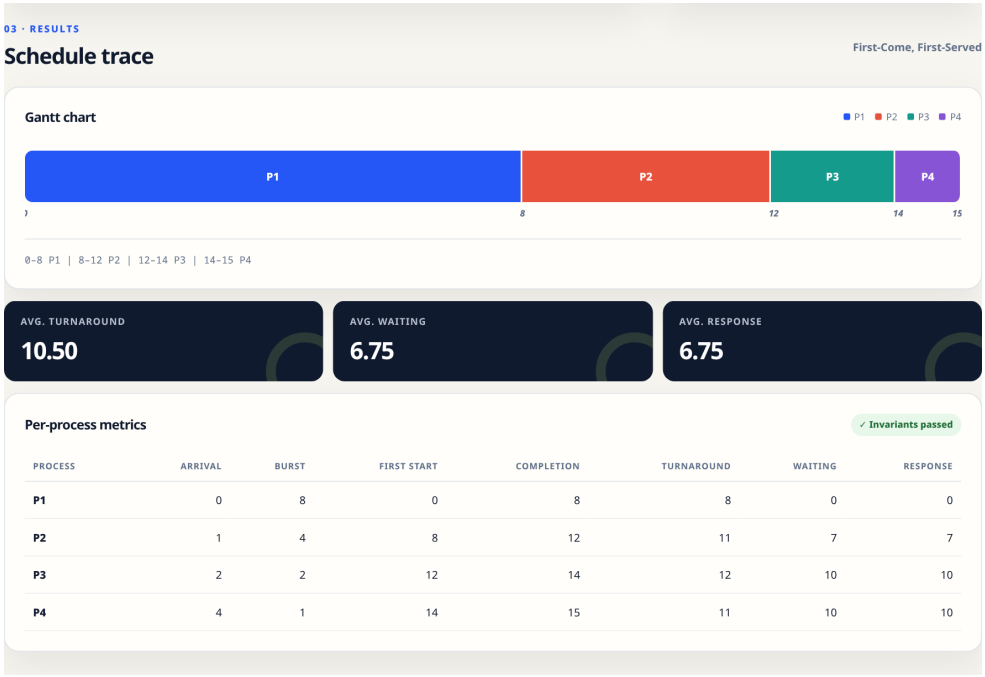


Figure 1: Simulator output for FCFS on Workload A.

2.2 SJF on Workload A

Input: P1=(0, 8), P2=(1, 4), P3=(2, 2), P4=(4, 1).

Hand decision trace. Non-preemptive P1 must finish. At time 8 all other jobs are ready, so ascending burst length gives P4 (1), P3 (2), and P2 (4).

```
0-8 P1  |  8-9 P4  |  9-11 P3  |  11-15 P2
```

Table 3: SJF Workload A: hand calculation and simulator output (identical).

Process	A	B	FS	C	T	W	R
P1	0	8	0	8	8	0	0
P2	1	4	11	15	14	10	10
P3	2	2	9	11	9	7	7
P4	4	1	8	9	5	4	4
			Average		**9.00**	**5.25**	**5.25**

Simulator comparison. The simulator produced the same timeline and every per-process value in the table. **Exact match; no discrepancy.**

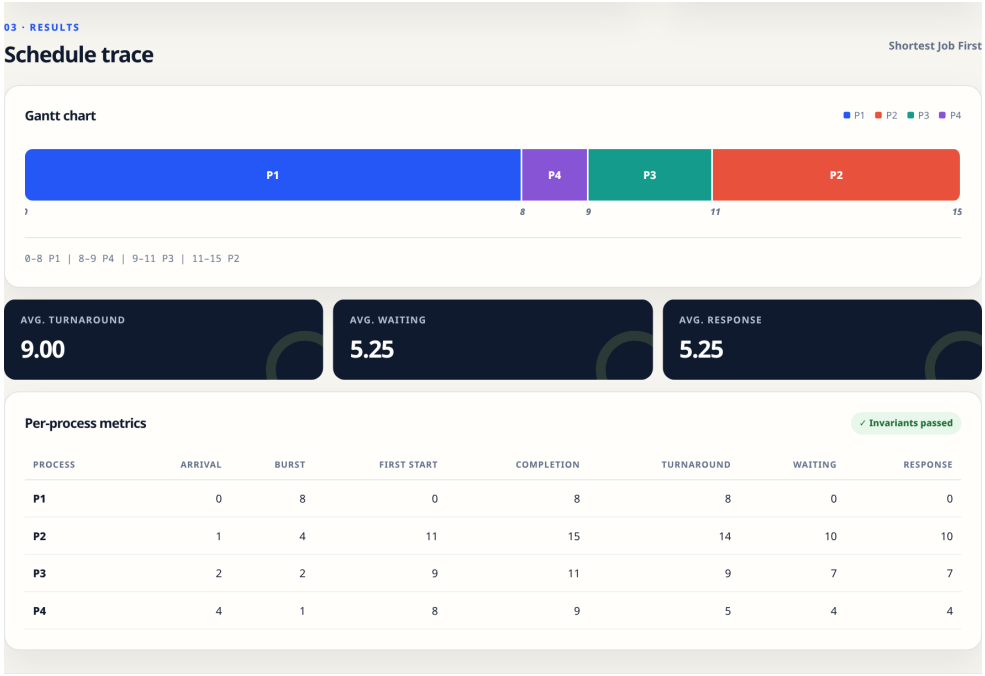


Figure 2: Simulator output for SJF on Workload A.

2.3 SRTF on Workload A

Input: P1=(0, 8), P2=(1, 4), P3=(2, 2), P4=(4, 1).

Hand decision trace. P2 arrives at time 1 with burst 4, which is shorter than P1's remaining 7, so it preempts P1. At time 2, P3's burst 2 is shorter than P2's remaining 3. P3 finishes at time 4; P4 then has the shortest remaining time. P2 resumes before P1.

```
0-1 P1  |  1-2 P2  |  2-4 P3  |  4-5 P4  |  5-8 P2  |  8-15 P1
```

Table 4: SRTF Workload A: hand calculation and simulator output (identical).

Process	A	B	FS	C	T	W	R
P1	0	8	0	15	15	7	0
P2	1	4	1	8	7	3	0
P3	2	2	2	4	2	0	0
P4	4	1	4	5	1	0	0
			Average		**6.25**	**2.50**	**0.00**

Simulator comparison. The simulator produced the same timeline and every per-process value in the table. **Exact match; no discrepancy.**

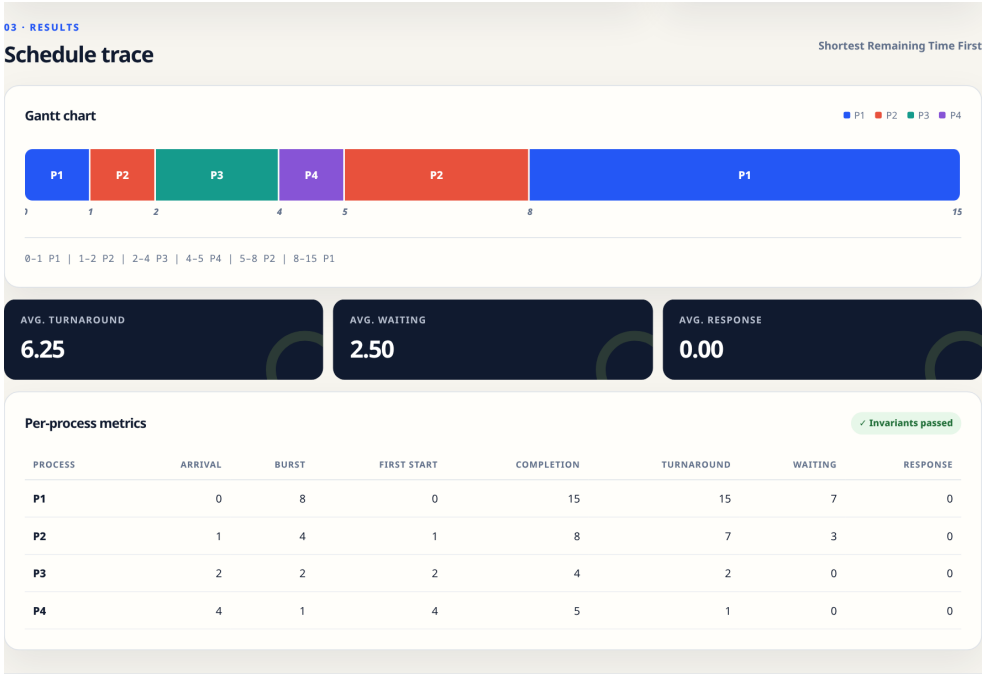


Figure 3: Simulator output for SRTF on Workload A.

2.4 Round Robin (q=2) on Workload A

Input: P1=(0, 8), P2=(1, 4), P3=(2, 2), P4=(4, 1); quantum $q = 2$.

Hand decision trace. Each process at the queue head receives at most two units. P3's arrival at time 2 is enqueued before P1 is returned to the tail. P4 similarly enters at the time-4 boundary before P2 is re-enqueued. The final two P1 slices remain separately visible even though they are consecutive.

```
0-2 P1 | 2-4 P2 | 4-6 P3 | 6-8 P1 | 8-9 P4 | 9-11 P2 | 11-13 P1 | 13-15 P1
```

Table 5: Round Robin Workload A: hand calculation and simulator output (identical).

Process	A	B	FS	C	T	W	R
P1	0	8	0	15	15	7	0
P2	1	4	2	11	10	6	1
P3	2	2	4	6	4	2	2
P4	4	1	8	9	5	4	4
			Average		**8.50**	**4.75**	**1.75**

Simulator comparison. The simulator produced the same eight slices and every per-process value in the table. **Exact match; no discrepancy.**

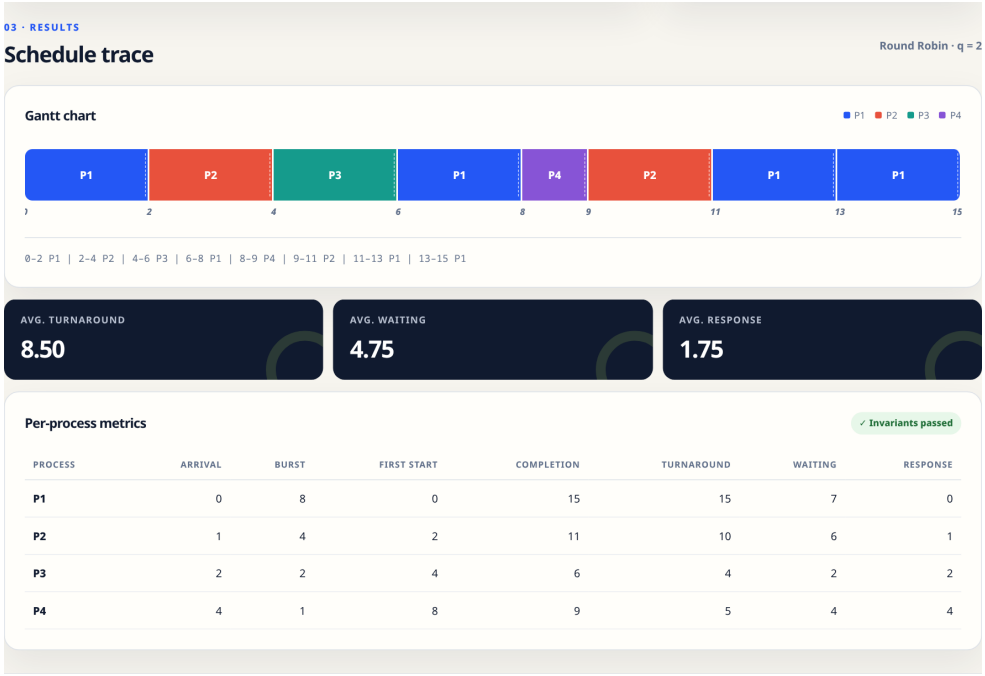


Figure 4: Simulator output for Round Robin ($q = 2$) on Workload A.

3 Worked examples: Workload B

3.1 FCFS on Workload B

Input: P1=(2, 3), P2=(2, 1), P3=(8, 2), P4=(9, 1).

Hand decision trace. The CPU is idle until time 2. P1 and P2 arrive together, so earlier input position makes P1 first. The CPU becomes idle again at time 6. P3 begins at time 8 and P4 waits until P3 completes.

```
0-2 IDLE | 2-5 P1 | 5-6 P2 | 6-8 IDLE | 8-10 P3 | 10-11 P4
```

Table 6: FCFS Workload B: hand calculation and simulator output (identical).

Process	A	B	FS	C	T	W	R
P1	2	3	2	5	3	0	0
P2	2	1	5	6	4	3	3
P3	8	2	8	10	2	0	0
P4	9	1	10	11	2	1	1
			Average		**2.75**	**1.00**	**1.00**

Simulator comparison. The simulator produced the same running and idle intervals and every per-process value in the table. **Exact match; no discrepancy.**

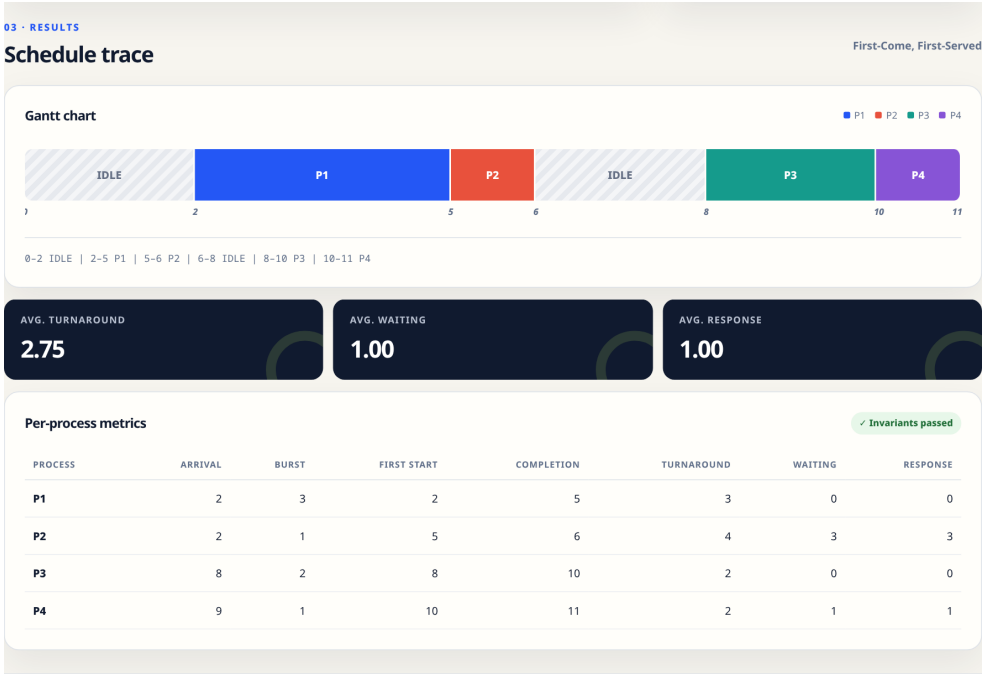


Figure 5: Simulator output for FCFS on Workload B.

3.2 SJF on Workload B

Input: P1=(2, 3), P2=(2, 1), P3=(8, 2), P4=(9, 1).

Hand decision trace. At time 2, P2 has the shortest burst, then P1 runs. At time 8 only P3 is ready, and non-preemptive SJF does not interrupt it when P4 arrives.

```
0-2 IDLE | 2-3 P2 | 3-6 P1 | 6-8 IDLE | 8-10 P3 | 10-11 P4
```

Table 7: SJF Workload B: hand calculation and simulator output (identical).

Process	A	B	FS	C	T	W	R
P1	2	3	3	6	4	1	1
P2	2	1	2	3	1	0	0
P3	8	2	8	10	2	0	0
P4	9	1	10	11	2	1	1
			Average		**2.25**	**0.50**	**0.50**

Simulator comparison. The simulator produced the same timeline and every per-process value in the table. **Exact match; no discrepancy.**

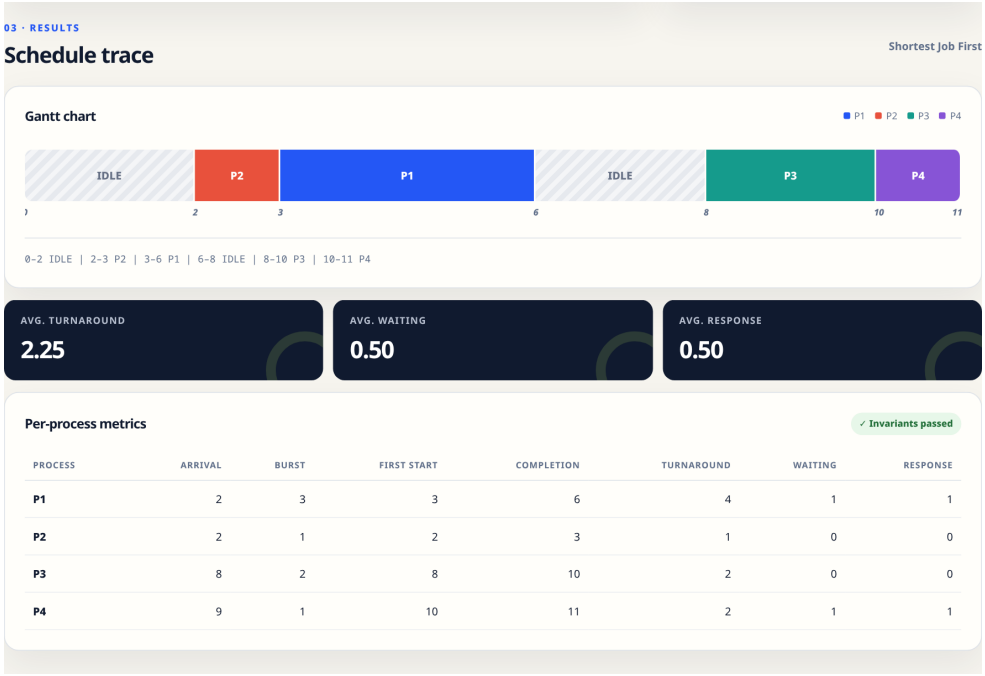


Figure 6: Simulator output for SJF on Workload B.

3.3 SRTF on Workload B

Input: P1=(2, 3), P2=(2, 1), P3=(8, 2), P4=(9, 1).

Hand decision trace. P2 runs before P1 at time 2. At time 9, P3 has one unit remaining and the newly arrived P4 also requires one unit. Equal remaining time does not cause preemption, so P3 continues. The schedule is identical to SJF.

```
0-2 IDLE | 2-3 P2 | 3-6 P1 | 6-8 IDLE | 8-10 P3 | 10-11 P4
```

Table 8: SRTF Workload B: hand calculation and simulator output (identical).

Process	A	B	FS	C	T	W	R
P1	2	3	3	6	4	1	1
P2	2	1	2	3	1	0	0
P3	8	2	8	10	2	0	0
P4	9	1	10	11	2	1	1
			Average		**2.25**	**0.50**	**0.50**

Simulator comparison. The simulator produced the same timeline and every per-process value in the table. **Exact match; no discrepancy.**

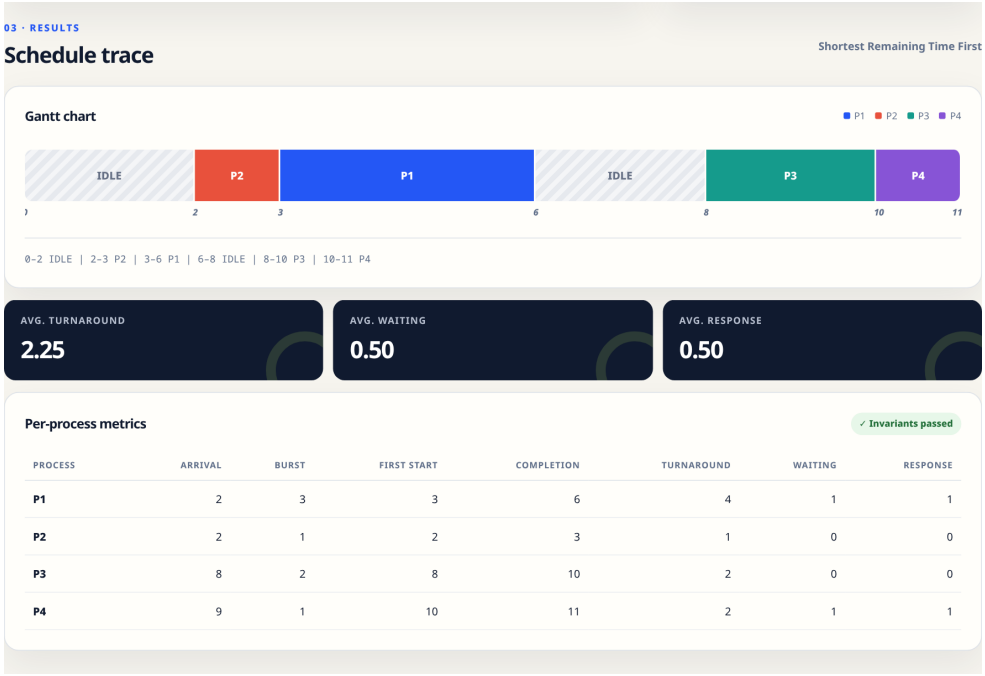


Figure 7: Simulator output for SRTF on Workload B.

3.4 Round Robin (q=2) on Workload B

Input: P1=(2, 3), P2=(2, 1), P3=(8, 2), P4=(9, 1); quantum $q = 2$.

Hand decision trace. P1 and P2 arrive together, so input order places P1 first. After its two-unit slice, P2 runs before re-enqueued P1. P3 completes its two-unit slice even though P4 arrives at time 9.

```
0-2 IDLE | 2-4 P1 | 4-5 P2 | 5-6 P1 | 6-8 IDLE | 8-10 P3 | 10-11 P4
```

Table 9: Round Robin Workload B: hand calculation and simulator output (identical).

Process	A	B	FS	C	T	W	R
P1	2	3	2	6	4	1	0
P2	2	1	4	5	3	2	2
P3	8	2	8	10	2	0	0
P4	9	1	10	11	2	1	1
			Average		**2.75**	**1.00**	**0.75**

Simulator comparison. The simulator produced the same timeline and every per-process value in the table. **Exact match; no discrepancy.**

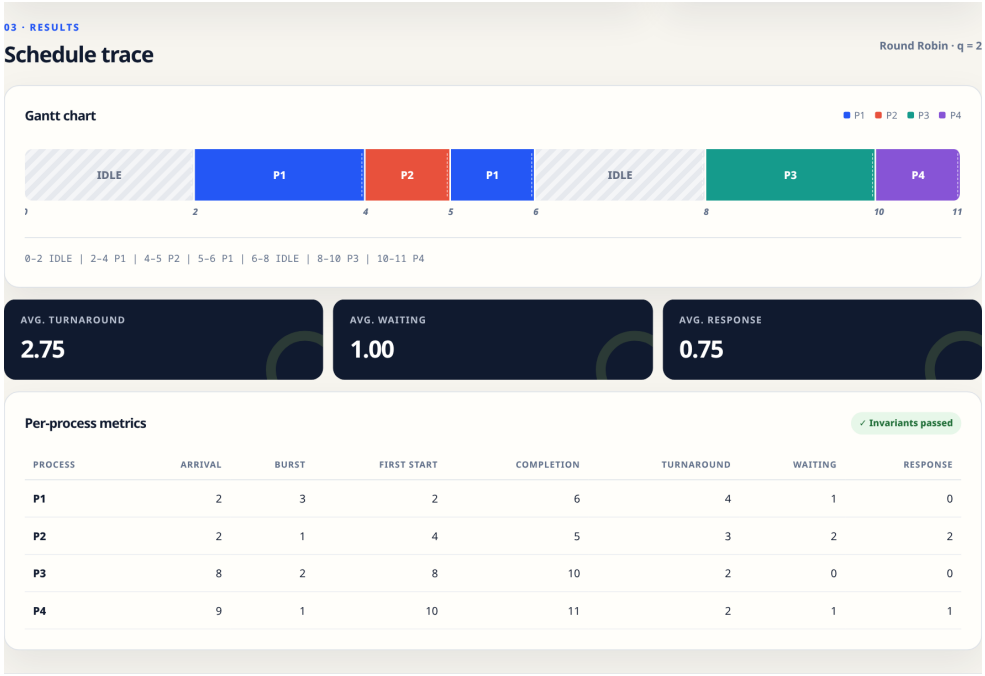


Figure 8: Simulator output for Round Robin ($q = 2$) on Workload B.

4 Additional test cases

4.1 Idle CPU test

Input: A=$(2, 2)$ and B=$(6, 1)$; algorithm FCFS.

Behavior checked: The initial and middle idle intervals must be visible, and the CPU must restart as soon as the next process arrives.

```
Expected:  0-2 IDLE  |  2-4 A  |  4-6 IDLE  |  6-7 B
```

Expected A: $FS = 2$, $C = 4$, $T = 2$, $W = 0$, $R = 0$. Expected B: $FS = 6$, $C = 7$, $T = 1$, $W = 0$, $R = 0$. The expected averages are $T = 1.50$, $W = 0$, and $R = 0$. The simulator matched these results.

4.2 Simultaneous arrivals and scheduling tie

Input: P1=$(0, 2)$, P2=$(0, 2)$, P3=$(1, 1)$; algorithm SJF.

Behavior checked: P1 and P2 tie on arrival and burst, so earlier input position selects P1. When it finishes, the shorter P3 runs before P2.

```
Expected:  0-2 P1  |  2-3 P3  |  3-5 P2
```

Expected (T, W, R): P1=$(2, 0, 0)$, P2=$(5, 3, 3)$, P3=$(2, 1, 1)$. The expected averages are $T = 3.00$, $W = 1.33$, and $R = 1.33$. The simulator matched these results.

4.3 SRTF preemption and equal-remaining rule

Input: P1=$(0, 7)$, P2=$(2, 2)$, P3=$(3, 1)$; algorithm SRTF.

Behavior checked: P2 strictly preempts P1 at time 2. At time 3, P2 and newly arrived P3 both have one unit remaining, so P2 must continue rather than be preempted.

```
Expected:  0-2 P1  |  2-4 P2  |  4-5 P3  |  5-10 P1
```

Expected (T, W, R): P1=$(10, 3, 0)$, P2=$(2, 0, 0)$, P3=$(2, 1, 1)$. The expected averages are $T = 4.67$, $W = 1.33$, and $R = 0.33$. The simulator matched these results.

4.4 Verified schedule properties

Property	Verification method	**Result**
Exact CPU burst	Sum every process interval and compare it with the requested burst.	**Pass**
No execution before arrival	Require every process interval start to be at or after its arrival.	**Pass**
No CPU overlap	Require each interval to start exactly when the previous interval ends.	**Pass**
Nonnegative metrics	Check every waiting and response value.	**Pass**
Work conserving	For every idle interval, confirm that no arrived process has unfinished work.	**Pass**

The automated test suite applies these invariants to all four algorithms on both required workloads. Exact-output tests also cover ties, Round Robin boundary ordering, preserved RR slices, and rejected invalid input.

5 Analysis

5.1 Which algorithm is best for Workload A?

Algorithm	Average T	Average W	Average R
FCFS	10.50	6.75	6.75
SJF	9.00	5.25	5.25
SRTF	6.25	**2.50**	**0.00**
RR ($q = 2$)	8.50	4.75	1.75

SRTF gives both the lowest average waiting time (2.50) and the lowest average response time (0.00). Each later job in A is shorter than the older job's remaining time when it first runs, so it preempts and receives an immediate first response. This favors the short arrivals while P1 absorbs most of the waiting. SJF knows the burst lengths but cannot interrupt P1's initial eight-unit burst.

5.2 Round Robin with quantum 1, 2, and 4

Quantum	Slices	Average T	Average W	Average R
1	15	8.50	4.75	0.75
2	8	8.50	4.75	1.75
4	5	9.25	5.50	3.75

For Workload A, $q = 1$ improves initial responsiveness but does not improve average turnaround or waiting relative to $q = 2$. A smaller quantum moves first starts earlier, but it also interrupts jobs more often and may postpone completion through additional queue rotations. This simulator assumes zero context-switch cost. On a real CPU, an extremely small quantum would additionally lose useful time to context switching. Therefore, decreasing the quantum is not guaranteed to improve every metric.

5.3 When SJF and SRTF produce the same schedule

SJF and SRTF produce the same schedule when, after each dispatch, no later arrival has a burst strictly shorter than the running process's remaining time. This includes a batch where all jobs are already available, or later arrivals that cannot justify preemption. Workload B is an example. At time 9, P4 arrives while P3 has one unit remaining. P4 also needs one unit, so SRTF's equality rule keeps P3 running and the SJF and SRTF schedules remain identical.

5.4 Why real operating systems estimate CPU bursts

The simulator receives the complete synthetic workload in advance, including the exact duration of every future CPU burst. A real operating system cannot know how long a program's next CPU burst will last because execution depends on inputs, branches, cache behavior, I/O, and future events. A practical scheduler must use estimates from observed history or indirect priority and responsiveness signals. This simulator can make ideal SJF/SRTF decisions only because the input gives it perfect future knowledge.

5.5 Starvation under SJF and SRTF

Starvation can occur under both policies. Under SJF, a ready long job may repeatedly lose dispatch opportunities if shorter jobs keep arriving before the CPU becomes free. Under SRTF, a long job may be delayed or preempted whenever strictly shorter work arrives. The finite workloads in this assignment eventually stop producing new jobs, so every process finishes. This removes the unbounded-arrival condition needed for indefinite starvation. Aging is one possible mitigation in a real scheduler.

5.6 Implementation mistake exposed by testing

The first browser render showed the Round Robin quantum field even while FCFS was selected. The HTML correctly used the `hidden` attribute, but the component CSS rule set `display:grid` and overrode the browser's default hidden style. A headless-browser screenshot exposed the problem. I added an explicit `[hidden] { display:none !important; }` rule and repeated the visual check. The scheduling-engine tests alone would not have detected this UI-only error.

6 Implementation notes

The scheduling engine is a pure JavaScript module separated from the page renderer. FCFS and SJF dispatch whole jobs. SRTF advances one integer time unit so arrivals and the strict-preemption rule are evaluated exactly. Round Robin maintains an explicit FIFO queue, processes arrivals through each boundary, and only then re-enqueues an unfinished runner. Non-RR adjacent intervals are merged for chart readability; RR slices are intentionally kept separate.

Invalid input produces specific messages for missing or duplicate IDs, invalid arrival/burst values, unsupported algorithms, and an invalid Round Robin quantum. The site requires no backend and stores no personal data. It models the assignment policies and does not reproduce Ubuntu's actual scheduler.

7 AI assistance statement

I used OpenAI Codex to help implement the HTML/CSS/JavaScript interface and pure scheduling module, draft automated tests, check arithmetic, debug the UI, and organize this report. I checked the generated work by independently tracing Workloads A and B, comparing every timeline and metric, running exact-output and invariant tests, and rendering the interface in a headless browser.

I rejected one generated-code optimization for Round Robin: the general timeline helper normally merges adjacent intervals belonging to the same process. Doing so would hide a quantum boundary when no other process is ready, contrary to the assignment. Round Robin therefore records consecutive slices separately.